



Islamic University of Technology

Board Bazar, Gazipur, Dhaka

SWE 4304 – Software Project Lab-I

Final Project Report

**Virtual File System
Simulator**

March 30, 2026

SPL Team - 11

Supervisor:

Samnun Azfar
Junior Lecturer
Islamic University of Technology

Team Members:

Ashfaque Sadad
ID: 230042102
Bsc. in Software Engineering
Department of Computer Science and Engineering

Hasan Mobarak Mahi
ID: 230042129
Bsc. in Software Engineering
Department of Computer Science and Engineering

Zuhaaer Mustafa Sadab
ID: 230042132
Bsc. in Software Engineering
Department of Computer Science and Engineering

Chapter 1

Problem Statement & Potential Solution

1.1 Problem Statement

Understanding how file systems work internally is a fundamental concept in computer science and operating systems. However, students rarely get the opportunity to interact with or observe the inner workings of a real file system due to the complexity and risk involved in modifying actual operating system components. The core problems this project addresses are:

- Modern operating systems abstract away all file system operations, making it difficult for students to understand how files are physically stored, retrieved, and organized on a disk.
- There is no safe, hands-on environment where students can experiment with low-level file system concepts such as inode allocation, bitmap-based block management, superblock metadata, and directory entry structures without risking damage to a real system.
- The relationship between logical file operations (creating, reading, writing, deleting files) and their physical representation on disk (block allocation, inode updates, bitmap modifications) remains largely invisible to the end user.

1.2 Potential Solution

To address these problems, we developed an EXT-2 Virtual File System Simulator — a fully functional, command-line based file system that operates on a binary disk image file (disk.img) instead of a real hard drive. The solution provides:

- A simulated disk environment where all file system operations are performed safely within a single binary file, eliminating any risk to the host operating system.
- A complete implementation of core EXT-2 file system concepts including a Superblock for metadata management, dual Bitmaps for

tracking inode and block allocation, an Inode Table for file metadata, and Data Blocks for actual file content storage.

- An interactive shell interface that accepts standard Linux-style commands (mkdir, touch, ls, cd, read, write, rm, cp, mv, find), allowing users to interact with the virtual file system in a familiar and intuitive manner.
- Additional features such as comprehensive operation logging with timestamps, and recursive file search to demonstrate real-world file system capabilities beyond basic CRUD operations.

Chapter 2

Implemented Features

The following features have been fully implemented and tested in the project:

Core File System Operations

- **Format:** Complete disk initialization with Superblock, Inode Bitmap, Block Bitmap, Inode Table, and Data Block regions.
- **File Creation and Deletion:** Create empty files (touch) and permanently delete files (rm) with proper inode and block deallocation.
- **File Read and Write:** Write text content to files and read it back. Data is stored across 1 KB data blocks with automatic block allocation.
- **Directory Management:** Create directories (mkdir), list contents (ls), navigate into and out of directories (cd), and display the current path (pwd).

Advanced File Operations

- **Copy (cp):** Create independent duplicates of files with new inodes and data blocks. Supports same-folder copy, cross-directory copy, and copy with rename.

- **Move (mv):** Relocate or rename files and directories. Operates by updating directory entries without physically moving data blocks. Includes automatic rollback on failure.
- **Recursive Search (find):** Depth-First Search across the entire file system from the root directory. Supports filtering by file type (-f) or directory type (-d).

System Utilities

- **Disk Statistics (info):** Real-time display of inode and block usage pulled directly from the allocation bitmaps.
- **Operation Logging (log):** Every file system operation is timestamped and appended to a persistent log file (fs_log.txt). Supports viewing and clearing the log.
- **XOR Encryption (encrypt):** File content encryption using a repeating-key XOR cipher. Encryption and decryption occur transparently during write and read operations.

Architecture and Design

- Modular class-based architecture with 11 separate manager classes, each handling a single responsibility.
- Centralized constants file (Constants.h) for all file system parameters.
- Path resolution support for multi-level relative paths in cp and mv commands.
- Input validation for filenames and paths using PathParser.

Chapter 3

Tools and Technology

Category	Tool / Technology
Programming Language	C++ (C++11 Standard)
Compiler	GNU g++ (MinGW on Windows)
Build System	GNU Make (Makefile) and Windows Batch Script
Version Control	Git and GitHub
IDE / Editor	Visual Studio Code
Binary I/O	C++ fstream (ifstream, ofstream)
Libraries	C++ Standard Library only (no external dependencies)
Disk Format	Custom binary format inspired by the Linux EXT-2 file system

Chapter 4

Application Domain

This project falls under the domain of Systems Programming and Operating Systems. Specifically, it serves as an educational tool for understanding:

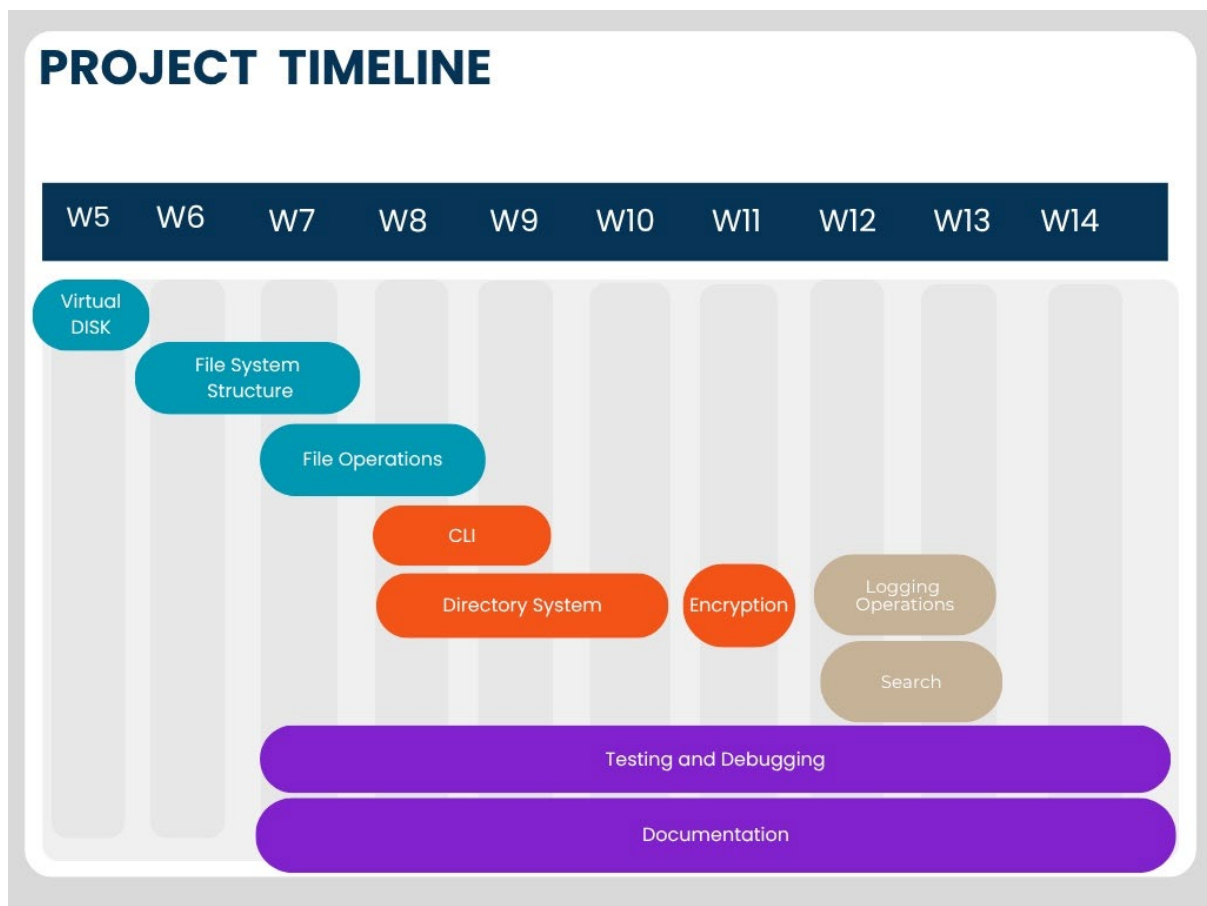
- File system internals and disk management concepts.
- How operating systems organize, store, and retrieve data at the block level.
- The role of metadata structures (Superblock, Inodes, Bitmaps) in managing a file system.
- Practical application of data structures (arrays, bitmaps, structs) in systems-level software.

The application domain of this project lies in systems-level software development, specifically in building and understanding storage

management systems that handle data organization, allocation, and retrieval at the block level..

Chapter 5

Project Timeline



Chapter 6

Contribution

Ashfaq Sadad (ID-230042102) — Core Structures & Virtual Disk

- **VirtualDisk Module:** Designed and implemented the VirtualDisk class, which serves as the lowest layer of the entire system. This class manages the creation, opening, and closing of the binary disk image file (disk.img) and provides block-level read and write

operations using C++ fstream. All other modules access the disk exclusively through this class.

- **SuperblockManager Module:** Developed the SuperblockManager class responsible for initializing, loading, saving, and validating the Superblock — the master metadata record stored in Block 0. This module calculates and stores the byte offsets of all disk regions (Inode Bitmap, Block Bitmap, Inode Table, Data Blocks) during formatting, and provides getter and setter methods used by every other manager.
- **BitmapManager Module:** Implemented the BitmapManager class that provides bit-level tracking of free and used resources. Two instances are used in the system — one for tracking inode allocation and one for tracking data block allocation. The module supports allocating (finding the first free bit and flipping it), deallocating, and counting free/used slots.
- **BlockManager Module:** Built the BlockManager class that acts as the intermediary between the bitmap and the raw disk. It handles allocating and freeing data blocks, reading and writing data to specific block positions, and calculating the exact byte offset of any block on disk.
- **Constants Definition:** Defined and maintained the centralized Constants.h header file containing all file system parameters (block size, total blocks, total inodes, direct block count, maximum filename length, and the NULL_BLOCK sentinel value). This file is included by every module in the project to ensure consistency.
- **Build System:** Created and maintained the Makefile for GNU Make-based compilation and the build_and_run.bat batch script for easy one-click compilation and execution on Windows. These build files were updated throughout development as new source files were added.
- **GitHub Repository Management:** Served as the primary GitHub repository manager — responsible for creating and managing branches, reviewing and merging pull requests from team members, and maintaining the overall integrity and organization of the remote

codebase.

Zuhaaer Mustafa Sadab (ID-230042132) — Inode, File & Directory Engine

- **InodeManager Module:** Designed and implemented the InodeManager class responsible for all inode-related operations. This includes allocating new inodes with the correct type (file or directory), deallocating inodes when files are deleted, reading and writing inode structs to and from the Inode Table on disk, and updating inode metadata such as file size and timestamps.
- **FileManager Module:** Developed the FileManager class that provides the high-level API for all file content operations. This includes creating empty files, writing text content to files by splitting data across multiple 1 KB blocks, reading file content back by reassembling data from multiple blocks, appending new content to existing files, truncating files, and deleting files with proper deallocation of all associated data blocks and the inode.
- **DirectoryHandler Module:** Implemented the DirectoryHandler class that manages directory contents. Directories are stored as special files whose data blocks contain packed arrays of DirectoryEntry structs. This module handles creating new directories, adding entries (files or subdirectories) to a directory, removing entries, finding entries by name, listing all entries in a directory, checking if a directory is empty, and performing recursive Depth-First Search across the entire file system for the find command.
- **EncryptionManager Module:** Built the EncryptionManager class that provides XOR-based file content encryption and decryption. The module uses a repeating-key XOR cipher where each byte of file data is XOR'd with a cycling byte from the key string before being written to disk. Since XOR is its own inverse, the same operation is used for both encryption and decryption. The module supports encryption at runtime and changing the encryption key.
- **Team Coordination:** Coordinated team meetings and discussions throughout the project, ensuring regular communication among team

members and with the course supervisor regarding project progress, technical decisions, milestone tracking, and deadline management.

Hasan Mobarak Mahi (ID-230042129)— Shell, Navigation, Logging & Integration

- **Ext2Structs Definition:** Designed and implemented the core data structures file (Ext2Structs.h) containing the struct definitions used throughout the entire project. This includes the Superblock struct (filesystem metadata with padding to exactly fill one block), the Inode struct (file metadata with 12 direct block pointers, timestamps, and type information), and the DirectoryEntry struct (file/directory name, inode number, and type). These structs define the exact binary layout of data on disk.
- **PathParser Module:** Implemented the PathParser class providing static utility methods for manipulating and validating file paths. This includes splitting path strings into individual components, checking if paths are absolute, normalizing paths by resolving "." references, joining path components back together, extracting the parent directory and basename from paths, and validating individual filename components against length and character restrictions.
- **ShellHelper Module:** Built the ShellHelper class containing static methods for all console display formatting. This includes printing the startup banner when the program launches, displaying the help menu listing all available commands, formatting individual directory entries with their type indicators ([DIR] or [FILE]) for the ls command, and converting raw byte counts into human-readable size strings (e.g., "10 MB", "512 KB").
- **Logger Module:** Developed the Logger class responsible for recording all file system operations to a persistent log file (fs_log.txt). Every operation — whether successful or failed — is timestamped in YYYY-MM-DD HH:MM:SS format and written with the appropriate severity level (INFO, WARN, or ERROR). The module supports printing the full log to the terminal and clearing the log file.
- **Main Application & Interactive Shell:** Served as the primary maintainer of main.cpp — the central integration point of the entire

project. This involved implementing the command parser that tokenizes user input, building the interactive shell loop that dispatches commands to their respective handlers, implementing the directory navigation stack that enables the `cd` command to move forward and backward through the directory tree, developing the `resolvePath()` function that enables multi-level path resolution for the `cp` and `mv` commands, and connecting all 11 manager classes into a cohesive, user-facing command-line application.

Chapter 7

Future Work

While the current implementation provides a fully functional EXT-2 file system simulator, several enhancements could be pursued in future iterations:

- **Indirect Block Pointers:** The current implementation supports only 12 direct block pointers per inode, limiting file size to 12 KB. Adding single and double indirect block pointers would significantly increase the maximum file size.
- **File Permissions:** Implementing a basic permission system (read, write, execute) for files and directories to simulate access control.
- **Multi-level `cd` Command:** Extending the `cd` command to support multi-level paths (e.g., `cd docs/reports`) in a single command rather than requiring one level at a time.
- **Wildcard Support in `Find`:** Enhancing the search command to support partial name matching and wildcard patterns (e.g., `find *.txt`).
- **Disk Defragmentation:** Implementing a defragmentation tool that reorganizes fragmented file blocks into contiguous sequences for improved read performance.

Chapter 8

Source Repository

Github Repository: <https://github.com/AshfaqueSadam/virtual-file-system.git>